



Campus Santa Fe

**Database Normalization for the Project Supported by Generative AI
Resources**

Software Construction and Decision Making

Estudiante:

Jesús José Espinoza Torruco

Vladimir Piñera Reyes

Gonzalo Zamarrón Orrantia

Matrícula:

A01781963

A01786772

A01782739

Fecha de entrega:

26 de mayo

1. Introduction

This document presents the evolution of the Dimension Deck database from its original conceptual design (Version 1, 19 entities) to the normalized implementation (Version 2, 11 tables in 3NF). It explains which normalization decisions were based on theory covered in class, which improvements were suggested by a generative language model, and why normalization matters for data integrity and long-term maintainability.

2. Version 1 — Original Conceptual Schema (19 Entities)

The following schema represents the original entity-relationship design produced by the team before implementation. It captures all game concepts as separate entities and served as the starting point for normalization.

2.1 Version 1 DBML

```
Table user {
  id          integer [pk]
  username    varchar
  email       varchar
  password    varchar
  created_at  timestamp
  updated_at  timestamp
}

Table user_stats {
  user_id      integer [pk, ref: - user.id]
  runs_played  integer
  victories    integer
  enemies_slain integer
  time_played  float
}

Table dimension {
  id          integer [pk]
  name        varchar
  difficulty  float
  soundtrack  varchar
}

Table run {
  id          integer [pk]
  dimension_id integer [ref: > dimension.id]
  seed        integer
  status      varchar
  started_at  timestamp
}

Table run_player {
  id          integer [pk]
  run_id      integer [ref: > run.id]
  hp          integer
  max_hp      integer
  credits     integer
  active_slots integer
  auto_slots  integer
  resist_penalty float
}
```

```
Table run_log {
  id          integer [pk]
  run_id       integer [ref: > run.id]
  room_id      integer [ref: > room.id]
  event_type   varchar
  description  text
}

Table room {
  id          integer [pk]
  run_id       integer [ref: > run.id]
  type         varchar
  cleared      boolean
  width_px     integer
  height_px    integer
}

Table slot_upgrade {
  id          integer [pk]
  run_player_id integer [ref: > run_player.id]
  slot_type    varchar
  cost         integer
  new_capacity  integer
}

Table shop_item {
  id          integer [pk]
  run_id       integer [ref: > run.id]
  item_type    varchar
  item_ref_id  integer
  price        integer
  sold         boolean
}

Table enemy {
  id          integer [pk]
  archetype    varchar
  sprite_px    integer
  alive        boolean
}
```

```
Table swarm_enemy {
  id          integer [pk]
  enemy_id    integer [ref: - enemy.id]
  speed       float
  contact_dmg integer
  crowd_priority boolean
}
```

```
Table tank_enemy {
  id          integer [pk]
  enemy_id    integer [ref: - enemy.id]
  speed       float
  contact_dmg integer
  knockback_res float
  melee_dmg   integer
  armor       integer
}
```

```
Table ranged_enemy {
  id          integer [pk]
  enemy_id    integer [ref: - enemy.id]
  speed       float
  proj_speed  float
  fire_cooldown float
  min_distance integer
  accuracy    float
}
```

```
Table card {
  id          integer [pk]
  rarity      varchar
  category    varchar
  cooldown    float
}
```

```
Table card_effect {
  id          integer [pk]
  card_id     integer [ref: > card.id]
  effect_type varchar
  value       float
  duration    float
  apply_chance float
}
```

```

Table damage_card {
  id          integer [pk]
  card_id     integer [ref: - card.id]
  dmg_amount  integer
  dmg_type    varchar
  is_aoe      boolean
}

Table heal_card {
  id          integer [pk]
  card_id     integer [ref: - card.id]
  heal_amount integer
  over_time   boolean
}

Table defense_card {
  id          integer [pk]
  card_id     integer [ref: - card.id]
  def_amount  integer
}

Table deck_card {
  id          integer [pk]
  card_id     integer [ref: > card.id]
  slot_type   varchar
  level       integer
  is_starter  boolean
}

```

2.2 Problems Identified in Version 1

Problem	Entity/Column	Violation
card_effect uses generic effect_type + value + duration columns — one row can represent anything from a damage multiplier to a heal-over-time, making it impossible to enforce type validity	card_effect	1NF — non-atomic, multi-purpose columns
card.rarity is a free text field; sort order (common < rare < epic < legendary) is not enforceable in the DB	card.rarity	3NF — transitive dependency on an unmodeled concept
card.category and deck_card.slot_type duplicate the same classification string across rows with no referential enforcement	card, deck_card	3NF — repeating group with transitive dependency

Problem	Entity/Column	Violation
<code>user_stats</code> stores data fully derivable from run records — any update to <code>run</code> requires a manual sync to <code>user_stats</code>	<code>user_stats</code>	3NF — stored derived data (update anomaly risk)
<code>enemy</code> uses a single base table + three subtype tables with separate PKs — <code>speed</code> and <code>contact_dmg</code> are repeated in all three subtypes	<code>swarm_enemy</code> , <code>tank_enemy</code> , <code>ranged_enemy</code>	2NF — partial dependency on the parent key
<code>damage_card</code> , <code>heal_card</code> , <code>defense_card</code> are separate tables but <code>card</code> already has <code>category</code> to distinguish them — the split forces three-table JOINS for basic card reads	<code>damage_card</code> , <code>heal_card</code> , <code>defense_card</code>	Design redundancy
<code>room</code> and <code>dimension</code> store procedurally-generated content that is never queried after creation	<code>room</code> , <code>dimension</code>	Not a normalization issue — architectural misfit

3. Version 2 — Normalized Schema (11 Tables, 3NF)

3.1 Version 2 DBML

```

Table rarities {
  id          int [pk, increment]
  name        varchar(20) [unique, not null,
                        note: "common | uncommon | rare | epic | legendary"]
  display_order int [not null, default: 0]
}

Table card_subtypes {
  id          int [pk, increment]
  card_type   varchar(9) [not null, note: "active | automatic"]
  subtype     varchar(50) [not null]
  indexes {
    (card_type, subtype) [unique, name: "uq_type_sub"]
  }
}

Table cards {
  id          int [pk, increment]
  card_name   varchar(100) [unique, not null]
  subtype_id  int [not null, ref: > card_subtypes.id]
  rarity_id   int [not null, ref: > rarities.id]
  base_damage int [default: 0]
  base_heal   int [default: 0]
  cooldown_seconds float [default: 0]
  shop_cost   int [default: 0, note: "0 = starter"]
  description text
}

Table card_effect_params {
  card_id      int [pk, ref: - cards.id]
  effect_range int [null]
  spread       float [null]
  shield       int [null]
  invincibility float [null]
  trigger_event varchar(50) [null,
                        note: "on_kill | on_attack | on_hit_received | on_dash"]
  threshold    float [null]
  heal_pct     float [null]
  full_heal    tinyint(1) [default: 0]
  from_enemy   tinyint(1) [default: 0]
}

```

```

Table enemies {
  id          int          [pk, increment]
  name        varchar(50) [unique, not null]
  archetype   varchar(10) [not null,
                          note: "swarm | tank | ranged | boss"]
  dimension   varchar(15) [not null,
                          note: "dark_ages | old_west"]
  hp          int          [not null]
  speed       float        [not null]
  contact_dmg int          [default: 0]
  width_px    int          [default: 16]
  height_px   int          [default: 16]
  shoot_rate  float        [null]
  preferred_distance int    [null]
}

Table users {
  id          int          [pk, increment]
  username    varchar(50) [unique, not null]
  email       varchar(100) [unique, not null]
  password_hash varchar(255) [not null]
  created_at  timestamp    [default: `CURRENT_TIMESTAMP`]
}

Table runs {
  id          int          [pk, increment]
  user_id     int          [not null, ref: > users.id]
  started_at  timestamp    [default: `CURRENT_TIMESTAMP`]
  ended_at    timestamp    [null]
  status      varchar      [note: "active | victory | defeat"]
  rooms_cleared int        [default: 0]
  enemies_killed int       [default: 0]
  damage_dealt int         [default: 0]
  damage_taken int         [default: 0]
  credits_earned int       [default: 0]
  cards_collected int     [default: 0]
  score       int         [default: 0]
}

```

```

Table run_cards {
  id      int [pk, increment]
  run_id  int [not null, ref: > runs.id]
  card_id int [not null, ref: > cards.id]
}

Table run_player_state {
  run_id      int [pk, ref: - runs.id]
  active_slots int [not null, default: 3]
  auto_slots  int [not null, default: 4]
  credits     int [not null, default: 0]
}

Table shop_offerings {
  id      int [pk, increment]
  run_id  int [not null, ref: > runs.id]
  card_id int [not null, ref: > cards.id]
  price   int [not null]
  sold    tinyint [default: 0]
  offered_at timestamp [default: `CURRENT_TIMESTAMP`]
  indexes {
    (run_id, card_id) [unique, name: "uq_run_card"]
  }
}

Table run_log {
  id      int [pk, increment]
  run_id  int [not null, ref: > runs.id]
  event_type varchar [not null,
    note: "room_cleared | enemy_killed | card_acquired
           | slot_upgraded | shop_purchase
           | damage_dealt | damage_taken | heal"]
  value   int [not null, default: 0]
  card_id int [null, ref: > cards.id]
  logged_at timestamp [default: `CURRENT_TIMESTAMP`]
}

```

4. Normalization Applied

4.1 First Normal Form (1NF)

Violation found: card_effect in Version 1 used three generic columns (effect_type VARCHAR, value FLOAT, duration FLOAT) to represent every possible card effect. This

means a single row could represent a heal, a shield, a damage bonus, or a trigger event — all stored the same way with no type enforcement.

```
-- Version 1 (violates 1NF)
card_effect: { card_id=7,  effect_type='full_heal', value=1,  duration=NULL }
card_effect: { card_id=11, effect_type='shield',   value=35, duration=NULL }
card_effect: { card_id=14, effect_type='trigger',  value=8,  duration=NULL }
```

Fix applied: Each distinct effect attribute became its own typed column in `card_effect_params`. The database can now enforce that shield is always an INT, invincibility is always a FLOAT in seconds, and trigger_event is always one of the valid string values.

```
-- Version 2 (1NF compliant)
card_effect_params: { card_id=11, shield=35, invincibility=1.5, ... }
card_effect_params: { card_id=14, shield=8,  trigger_event='on_attack', ... }
```

4.2 Second Normal Form (2NF)

Violation found: The three enemy subtype tables (`swarm_enemy`, `tank_enemy`, `ranged_enemy`) each repeated the columns `speed` and `contact_dmg`, which are attributes of the enemy concept, not of the specific subtype. A `swarm_enemy` row could not exist without its enemy row, yet the movement-related attributes were split across both.

Fix applied: All enemy attributes were consolidated into a single `enemies` table. Subtype-specific columns (`shoot_rate`, `preferred_distance`) are nullable and only populated for the relevant archetype. This eliminates the partial dependency while keeping the schema flat and queryable without multi-table joins.

4.3 Third Normal Form (3NF)

Violation 1 — `card.rarity`: The rarity string in each card row transitively implied a display order (`common=1`, `rare=3`, `legendary=5`). Any query that needed to sort by rarity tier had to implement that logic externally. A change to the ordering would require updating every card row.

Fix applied: `rarities` lookup table with `display_order INT`. Cards now store only `rarity_id INT`. Sort order is queried once from the lookup table.

Violation 2 — `card.category` + `deck_card.slot_type`: The card type (`active/automatic`) and subtype (`melee/heal/defense/drain`) were stored as free-text strings. Nothing prevented inserting `category='invalid'` or combining incompatible `type+subtype` pairs.

Fix applied: `card_subtypes` table with a `UNIQUE KEY (card_type, subtype)`. Only the combinations seeded into this table are valid. Trying to insert a card with an unknown `subtype_id` fails at the FK constraint.

Violation 3 — `user_stats`: All columns in this table (`runs_played`, `victories`, `enemies_slain`, `time_played`) are fully derivable from aggregating the run table. Storing them creates an update anomaly — every time a run ends, two tables must be updated atomically or they go out of sync.

Fix applied: `user_stats` was removed. Five analytical views (`v_user_run_summary`, `v_leaderboard`, `v_card_usage`, `v_top_cards`, `v_global_stats`) compute these values on demand from the `runs` and `run_cards` tables. The data is always consistent because it is never duplicated.

5. What Was Normalized by Class Concepts vs. AI Recommendations

5.1 Applied from class theory

Change	Normal Form	Principle
<code>card_effect</code> generic columns → <code>card_effect_params</code> typed columns	1NF	Each column must hold atomic, single-type data
Enemy subtype tables merged into <code>enemies</code> with nullable columns	2NF	No attribute should depend on a partial key
<code>card.rarity</code> string → <code>rarities</code> lookup with <code>display_order</code>	3NF	Non-key attribute (sort order) must not depend on another non-key attribute
<code>card.category</code> + subtype → <code>card_subtypes</code> lookup	3NF	Remove transitive dependency on valid type combinations
<code>user_stats</code> stored table → computed views	3NF	Derived data must not be stored redundantly
<code>run_player</code> + <code>run</code> merged into <code>runs</code> with <code>user_id</code> FK	2NF	Eliminated unnecessary junction for a simple 1:N relationship

5.2 Modifications suggested by the generative language model

The model was queried with a text description of the entities and relationships and asked for recommendations on constraints and structural improvements:

Recommendation	Rationale provided
Add CHECK constraints on <code>run_player_state</code> (<code>active_slots BETWEEN 3 AND 5, credits >= 0</code>)	Enforces game mechanics at DB level; prevents invalid states from being written by any client
Add UNIQUE KEY (<code>run_id</code> , <code>card_id</code>) to <code>shop_offerings</code>	Prevents duplicate cards in one shop offering under concurrent requests — application-level code alone cannot guarantee this
Use <code>ON DELETE SET NULL</code> for <code>run_log.card_id</code>	Audit/log entries should survive the deletion of a catalog card; only the reference is lost, not the historical event
Separate <code>run_player_state</code> as a 1:1 extension of <code>runs</code>	Keeps <code>runs</code> focused on outcome aggregates; the loadout state table changes at a different frequency
Flatten enemy subtypes into one table with nullable columns	At the current data scale (< 50 enemies), sparse nullable columns outperform the complexity of three subtype tables with multi-table joins

Important: Every AI recommendation was validated against the actual frontend code before being accepted. Slot bounds (3–5 active, 4–8 auto) come from `ShopItems.js` constants. Enemy stats come from `TankEnemy.js`, `SwarmEnemy.js`, `RangedEnemy.js`, and `BossEnemy.js`. The model provided structural patterns; the team verified the domain values.

6. Complete Version Comparison

6.1 Table-by-table mapping

Version 1 Entity	Version 2 Equivalent	Transformation
user	users	Renamed; password → password_hash (hashed storage)
user_stats	(removed)	Replaced by 5 analytical views
dimension	(removed)	Procedurally generated in frontend — no DB needed
run	runs	Merged with run_player; added aggregate stat columns
run_player	run_player_state	Reduced to loadout state (slots + credits); merged player HP into run
run_log	run_log	Kept; event_type made into ENUM; description removed
room	(removed)	Procedurally generated — layout never queried after creation
slot_upgrade	(absorbed into run_player_state)	Current slot capacity replaces upgrade history
shop_item	shop_offerings	Renamed; added persistent FK to cards; added UNIQUE constraint
enemy + swarm_enemy + tank_enemy + ranged_enemy	enemies	Collapsed 4 tables into 1 with nullable archetype-specific columns
card	cards	Merged with damage_card, heal_card, defense_card; rarity → FK
card_effect	card_effect_params	Generic columns → typed columns per effect attribute

Version 1 Entity	Version 2 Equivalent	Transformation
damage_card	(merged into cards)	base_damage column in cards
heal_card	(merged into cards)	base_heal column in cards
defense_card	(merged into card_effect_params)	shield column
deck_card	run_cards	Simplified to junction table; level/starter flags removed (handled in JS)
—	rarities	New — extracted from card.rarity to enforce sort order (3NF)
—	card_subtypes	New — extracted from card.category to enforce valid type combinations (3NF)

6.2 Summary metrics

Metric	Version 1	Version 2
Tables	19	11
Normal form	Pre-1NF (JSON/generic columns)	3NF
Analytical data	Stored in user_stats	Computed via 5 views
CHECK constraints	0	6
UNIQUE constraints	2	7
ON DELETE rules defined	0	11 (per FK)
Enemy tables	4 (base + 3 subtypes)	1 (flat with nullable columns)
Card type tables	4 (base + 3 subtypes)	2 (cards + card_effect_params)

7. Database Integrity Constraints in Version 2

7.1 Referential Integrity

Foreign Key	ON DELETE	Reason
cards → rarities	RESTRICT	Cannot delete a rarity that cards depend on

Foreign Key	ON DELETE	Reason
cards → card_subtypes	RESTRICT	Cannot delete a subtype with active cards
card_effect_params → cards	CASCADE	Effect params have no meaning without their card
runs → users	CASCADE	All run data belongs to the user
run_cards → runs	CASCADE	Cards acquired in a deleted run are irrelevant
run_player_state → runs	CASCADE	State is run-specific
shop_offerings → runs	CASCADE	Offerings are run-specific
run_log → runs	CASCADE	Log entries belong to the run
run_log → cards	SET NULL	Historical events are preserved even if a card is deleted

7.2 Domain Constraints

```

active_slots BETWEEN 3 AND 5 -- max 2 slot upgrades from default 3
auto_slots   BETWEEN 4 AND 8 -- max 4 slot upgrades from default 4
credits      >= 0             -- credits cannot go negative
price        > 0              -- all purchasable items must cost something
hp           > 0              -- enemy must have positive HP
speed        > 0              -- enemy must be able to move

```

8. Importance of Normalization

8.1 Eliminating anomalies

Without normalization three types of data anomalies can corrupt a database:

- **Insertion anomaly:** In Version 1, a new rarity tier (e.g. "mythic") could not be defined without also creating a card with that rarity — the concept had no independent existence.
- **Update anomaly:** In Version 1, changing the display order of "epic" required updating every row in cards where rarity = 'epic'. In Version 2, one row in rarities is updated.

- **Deletion anomaly:** In Version 1, deleting all cards of a given subtype destroyed the knowledge that the subtype existed. In Version 2, `card_subtypes` retains that information independently.

8.2 Enforcing validity at the data layer

Application code can have bugs, be bypassed by direct DB access, or behave differently across game versions. A schema without constraints relies entirely on the application to keep data valid. The 6 CHECK constraints and 7 UNIQUE constraints in Version 2 mean the database itself rejects invalid states regardless of which client writes to it.

8.3 How the generative model contributed without replacing the developers

The model was used as a **consultation tool** during constraint design. Given a text description of the entities and their game-domain meanings, it surfaced constraint patterns that are easy to overlook when focused on feature delivery:

- It identified that `shop_offerings` needed a composite UNIQUE key to be safe under concurrent API requests — a scenario not obvious when reasoning about a single-user flow.
- It recommended `ON DELETE SET NULL` for audit tables, distinguishing between data that is *meaningless without its parent* (use CASCADE) and data that *records a historical fact* (use SET NULL).
- It suggested flattening the enemy subtype hierarchy, providing the trade-off analysis between sparse nullable columns and separate subtype tables at different data scales.

In every case, the model provided options and rationale. The developers made the final decision after verifying the recommendation against the actual game code — a step the model cannot perform on its own.